```
function [Nps_atmo_nodens,Nps_defo_nodens] =
ps_output_nodens(Nifgs,Nps_atmo,Nps_defo,Npsc_selections,final_model,defo_model_
flag,master_res,ref_height,output_format,dates,Btemp,do_aposteriori_sidelobe_mas
k,crop)

% Function to create output in ascii format (e.g, for use in Grass
% or ArcGIS.
%
% Input:    - Nifgs                        number of interferograms
%           - Nps_atmo                     number of ps after
%                                          atmosphere modeling
%           - Nps_defo                     number of ps after
%                                          deformation modeling
%           - Npsc_selections              number of psc selections
%           - final_model                  model used for unwrapped data
%           - defo_model_flag              deformation model flag
%           - master_res                   master res file
%           - ref_height                   height of reference ps
%                                          tov geoid
%           - output_format                output format
%           - dates                        dates of acquisitions
%           - Btemp                        temporal baselines
%           - do_aposteriori_sidelobe_mask a posteriori sidelobe
%                                          mask flag
%           - crop                         crop in master image geometry
%
% Output:   - Nps_atmo_nodens              number of ps after
%                                          atmosphere modeling
%                                          without densification
%           - Nps_defo_nodens              number of ps after
%                                          deformation modeling
%                                          without densification
%
% ------------------------------------------------------------------------
% File............: ps_output_nodens.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Author..........: Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
% v1.7.2.8, Freek van Leijen
% - a posteriori sidelobe mask for all crop sizes
% v1.7.2.12, Freek van Leijen
% - linelo,pixlo from crop
% - read input per buffer
% v1.7.2.16, Freek van Leijen
% - dynamic grid_size
%

global project_id orbit
global Npar_max Nlines Npixels az_spacing r_spacing m2ph max_mem_buffer


% ------------------------------------------------------------------------
% Initialize
% ------------------------------------------------------------------------
```

```
grid_size = ceil(max_mem_buffer/(8*8*Nifgs*Npixels));
%dynamic, to allow all pixels in range directions for any Nifgs
buffer = 1;

MAXITER = 100;
CRITERPOS = 1e-8;
ellipsoid = [6378137.0 6356752.3141]; %wgs84

[Npar,par_index,covar_index] = ps_model_definitions('Npar',final_model);
[ps_annotation,par_index,covar_index] =
ps_model_definitions('annotation',final_model);

switch output_format
  case 1 %wgs84, radar original, radar gis coordinates
    ps_ascii_format1 = ['%12.8f,%12.8f,' repmat('%g,',1,Npar_max+7) '%g\n'];
    ps_ascii_headers = [{'phi' 'lambda' 'height' 'az_cn' 'r_cn' ...
                         'az_cn_gis' 'r_cn_gis'} ps_annotation(1:Npar_max) ...
                         {'ens_coh' 'ens_coh_local' 'stc'}];
  case 2 %wgs84, radar original coordinates
    ps_ascii_format1 = ['%12.8f,%12.8f,' repmat('%g,',1,Npar_max+5) '%g\n'];
    ps_ascii_headers = [{'phi' 'lambda' 'height' 'az_cn' 'r_cn'}
ps_annotation(1:Npar_max) {'ens_coh' 'ens_coh_local' 'stc'}];
  case 3 %wgs84, radar gis coordinates
    ps_ascii_format1 = ['%12.8f,%12.8f,' repmat('%g,',1,Npar_max+5) '%g\n'];
    ps_ascii_headers = [{'phi' 'lambda' 'height' 'az_cn_gis' 'r_cn_gis'}
ps_annotation(1:Npar_max) {'ens_coh' 'ens_coh_local' 'stc'}];
  case 4 % radar gis coordinates
    ps_ascii_format1 = [repmat('%g,',1,Npar_max+4) '%g\n'];
    ps_ascii_headers = [{'az_cn_gis' 'r_cn_gis'} ps_annotation(1:Npar_max)
{'ens_coh' 'ens_coh_local' 'stc'}];
  case 5 % terrafirma S5
    [dummy,master_index] = max(Btemp(Btemp<0));
    %ps_ascii_format1 = ['%s,' repmat('%g,',1,Nifgs+8) '%g\n'];
    ps_ascii_format1 = ['%g,%12.8f,%12.8f,' repmat('%g,',1,Nifgs+6) '%g\n'];
    ps_ascii_headers = [{'CODE' 'EASTING' 'NORTHING' 'RANGE' 'AZIMUTH' ...
                         'HEIGHT' 'VEL' 'COHERENCE' 'ST_DEV'} ...
                         dates(1:master_index)' dates(Nifgs+1)
dates(master_index+1:Nifgs)'];
  case 6 %order, wgs84, radar original, radar gis coordinates
    ps_ascii_format1 = ['%g,%12.8f,%12.8f,' repmat('%g,',1,Npar_max+7) '%g\n'];
    ps_ascii_headers = [{'order' 'phi' 'lambda' 'height' 'az_cn' 'r_cn' ...
                         'az_cn_gis' 'r_cn_gis'} ps_annotation(1:Npar_max) ...
                         {'ens_coh' 'ens_coh_local' 'stc'}];
  otherwise
    error('The output format you requested is not specified yet.');
end

switch defo_model_flag
  case 'yes'
    results_id = [{'atmo'};{'defo'}];
    Nps = [Nps_atmo Nps_defo];
    Nresults = 2;
  case 'no'
    results_id = {'atmo'};
    Nps = Nps_atmo;
    Nresults = 1;
end


% --------------------------------------------------------------------
% Read master.res
% --------------------------------------------------------------------
```

```
[container,orbit_state] = metadataNew(master_res);
linelo = crop(1);
pixlo = crop(3);

Nps_nodens = zeros(Npsc_selections,Nresults);


% ------------------------------------------------------------------------
% Create grid
% ------------------------------------------------------------------------

display('Creating grid ...');

grid_size_az = floor(grid_size/az_spacing);
grid_size_r = floor(grid_size/r_spacing);
grid_az = 1:grid_size_az:Nlines;
grid_r = 1:grid_size_r:Npixels;
if grid_az(end)~=Nlines
  grid_az = [grid_az Nlines+1];
else
  grid_az(end) = grid_az(end)+1;
end
if grid_r(end)~=Npixels
  grid_r = [grid_r Npixels+1];
else
  grid_r(end) = grid_r(end)+1;
end
Ngrid_az = length(grid_az)-1;
Ngrid_r = length(grid_r)-1;


for z = 1:Npsc_selections

  for w = 1:Nresults

    % --------------------------------------------------------------------
    % Determine buffersize
    % --------------------------------------------------------------------

    Nps_buffer = floor(max_mem_buffer/(5*(2*Nifgs+2)*8));

    if (Nps_buffer>=Nps(z,w))
      Nps_buffer = Nps(z,w);
      Nbuffers = 1;
      Nps_rem_buffer = 0;
    else
      Nbuffers = floor(Nps(z,w)/Nps_buffer);
      Nps_rem_buffer = rem(Nps(z,w),Nps_buffer);
      if (Nps_rem_buffer > 0)
        Nbuffers = Nbuffers + 1;
      end
    end

    Nps_buffer_orig = Nps_buffer;


    % --------------------------------------------------------------------
    % Open files
    % --------------------------------------------------------------------

    ps_fid = fopen([project_id '_ps_results_' char(results_id(w)) '_sel' ...
                    num2str(z) '.raw'],'r');

    ps_covar_fid = fopen([project_id '_ps_results_covar_'
```

```
char(results_id(w)) ...
                        '_sel'  num2str(z) '.raw'],'r');

    ps_stc_fid = fopen([project_id '_ps_stc_' char(results_id(w)) '_sel' ...
                    num2str(z) '.raw'],'r');

    ps_ascii_fid = fopen([project_id '_ps_ascii_nodens_' char(results_id(w)) ...
                        '_sel' num2str(z) '.csv'],'w');

    for v = 1:length(ps_ascii_headers)
        fprintf(ps_ascii_fid,'%s',char(ps_ascii_headers(v)));
        if v==length(ps_ascii_headers)
            fprintf(ps_ascii_fid,'\n');
        else
            fprintf(ps_ascii_fid,',');
        end
    end

    ps_defo_fid = fopen([project_id '_ps_ascii_nodens_' char(results_id(w)) ...
                        '_sel' num2str(z) '_defo_timeseries.csv'],'w');
    ps_atmo_fid = fopen([project_id '_ps_ascii_nodens_' char(results_id(w)) ...
                        '_sel' num2str(z) '_atmo_timeseries.csv'],'w');
    ps_defo_format = [repmat('%g,',1,Nifgs-1) '%g\n'];
    ps_atmo_format = ps_defo_format;

    ps_sidelobe_fid = fopen([project_id '_aposteriori_sidelobes_nodens_' ...
                        char(results_id(w)) '_sel' num2str(z) '.raw'],'w');

    ps_nodens_fid = fopen([project_id '_ps_results_nodens_' ...
                        char(results_id(w)) '_sel' num2str(z) '.raw'],'w');


    ps_plh_inc_fid = fopen([project_id '_ps_plh_inc_nodens_' char(results_id(w))
...
                        '_sel' num2str(z) '.raw'],'w');

    %create dummy file content, for later writing at correct spot
    for v = 1:Nps(z,w)
      fwrite(ps_plh_inc_fid,ones(4,1),'double');
    end
    fclose(ps_plh_inc_fid);
    ps_plh_inc_fid = fopen([project_id '_ps_plh_inc_nodens_' char(results_id(w))
...
                        '_sel' num2str(z) '.raw'],'r+');


    % ---------------------------------------------------------------------
    % Read radar coordinates
    % ---------------------------------------------------------------------

    ps_az = repmat(NaN,Nps(z,w),1);
    ps_r = repmat(NaN,Nps(z,w),1);

    Nps_buffer = Nps_buffer_orig;
    Nps_count = 0;

    for v = 1:Nbuffers

      if (v == Nbuffers)&&(Nps_rem_buffer~=0)
        Nps_buffer = Nps_rem_buffer;
      end

      ps_data = fread(ps_fid,[6+Npar_max+3*Nifgs Nps_buffer],'double')';
```

```
      ps_az(Nps_count+1:Nps_count+Nps_buffer) = ps_data(:,2);
      ps_r(Nps_count+1:Nps_count+Nps_buffer) = ps_data(:,3);

      clear ps_data

      Nps_count = Nps_count+Nps_buffer;

    end


    % ---------------------------------------------------------------------
    % Creating index
    % ---------------------------------------------------------------------

    display('Creating index ...');

    az_index = repmat(NaN,Nps(z,w),1);
    r_index = repmat(NaN,Nps(z,w),1);

    for v = 1:Ngrid_az
      index = find(ps_az>=grid_az(v)&ps_az<grid_az(v+1));
      az_index(index) = v;
    end

    for v = 1:Ngrid_r
      index = find(ps_r>=grid_r(v)&ps_r<grid_r(v+1));
      r_index(index) = v;
    end


    % ---------------------------------------------------------------------
    % Read data
    % ---------------------------------------------------------------------

    for g = 1:Ngrid_az

      display(['Creating output for azimuth grid ' num2str(g) 'of'
num2str(Ngrid_az) '...']);

      index1 = find(az_index==g);
      Nindex = length(index1);

      if ~isempty(index1)
        ps_data = repmat(NaN,Nindex,3*Nifgs+Npar_max+6);
        ps_covar = repmat(NaN,Nindex,Npar_max*(Npar_max+1)/2);
        ps_stc = repmat(NaN,Nindex,1);

        for k = 1:Nindex
          fseek(ps_fid,((index1(k)-1)*(3*Nifgs+Npar_max+6))*8,-1);
          ps_data(k,:) = fread(ps_fid,[3*Nifgs+Npar_max+6 1],'double');
          fseek(ps_covar_fid,((index1(k)-1)*(Npar_max*(Npar_max+1)/2))*8,-1);
          ps_covar(k,:) = fread(ps_covar_fid,[Npar_max*(Npar_max+1)/2
1],'double')';
          fseek(ps_stc_fid,(index1(k)-1)*8,-1);
          ps_stc(k) = fread(ps_stc_fid,1,'double');
        end

        ps_order = ps_data(:,1);
        ps_index = find(ps_order~=-1);
        ps_order = ps_order(ps_index);
        ps_data = ps_data(ps_index,:);

        ps_az = ps_data(:,2);
        ps_r = ps_data(:,3);
```

```
        ps_param = ps_data(:,4:Npar_max+3);
        ps_ens_coh = ps_data(:,Npar_max+4);
        ps_ens_coh_local = ps_data(:,Npar_max+5);
        ps_sig2hat = ps_data(:,Npar_max+6);
        ps_phase_unw = ps_data(:,Npar_max+7:Nifgs+Npar_max+6);
        ps_defo_tot = ps_data(:,Nifgs+Npar_max+7:2*Nifgs+Npar_max+6);
        ps_atmo = ps_data(:,2*Nifgs+Npar_max+7:3*Nifgs+Npar_max+6);
        clear ps_data


        % ----------------------------------------------------------------------
        % Geocoding
        % ----------------------------------------------------------------------

        line = ps_az+linelo-1;
        pixel = ps_r+pixlo-1;
        height = ps_param(:,1)+ref_height;

        [norm_orbit,norm_orbit_line] = intrp_orbit(line,container,orbit_state);

        [xyz] = lph2xyz(line,pixel,height,ellipsoid,container,...
                        norm_orbit_line,MAXITER,CRITERPOS);

        phi_lam_h = xyz2ell(xyz,ellipsoid);


        % ----------------------------------------------------------------------
        % Calculate incidence angle
        % ----------------------------------------------------------------------

        dr = [norm_orbit_line(:,2)-xyz(:,1) ...
            norm_orbit_line(:,3)-xyz(:,2) ...
            norm_orbit_line(:,4)-xyz(:,3)];
        r1 = sqrt(sum(dr.^2,2));
        inc_angle = acos(sum(xyz.*dr,2)./(r1.*sqrt(sum(xyz.^2,2))));
        inc_angle_deg = 180*inc_angle/pi;


        % ----------------------------------------------------------------------
        % Side lobe detection
        % ----------------------------------------------------------------------

        switch do_aposteriori_sidelobe_mask %phase based
          case 'yes'

            az_min = grid_az(g);
            az_max = grid_az(g+1)-1;
            r_min = 1;
            r_max = Npixels;

            Nlines_buffer = az_max-az_min+1;
            Npixels_buffer = r_max-r_min+1;

            mrm_buffer = freadbk([project_id '_mrm.raw'],Nlines,'float32',...
                                 az_min,az_max,r_min,r_max);

            inc_array = repmat(NaN,Nlines_buffer,Npixels_buffer);
            inc_array(sub2ind([Nlines_buffer Npixels_buffer],...
                              ps_az-az_min+1,ps_r-r_min+1)) = inc_angle;
            dinc_array = inc_array(:,2:end)-inc_array(:,1:end-1);

            neighbor_index = find(~isnan(dinc_array));
            neighbor_index2 =
unique([neighbor_index;neighbor_index+Nlines_buffer]);
```

```
            r1_array = sparse(Nlines_buffer,Npixels_buffer);
            r1_array(sub2ind([Nlines_buffer Npixels_buffer],...
                             ps_az-az_min+1,ps_r-r_min+1)) = r1;
            height_array = repmat(NaN,Nlines_buffer,Npixels_buffer);
            height_array(sub2ind([Nlines_buffer Npixels_buffer],...
                                 ps_az-az_min+1,ps_r-r_min+1)) = ps_param(:,1);
            dheight_array = height_array(:,2:end)-height_array(:,1:end-1);

            dheight_ref = repmat(NaN,Nlines_buffer,Npixels_buffer-1);
            dheight_ref(neighbor_index) =
-r1_array(neighbor_index).*sin(inc_array(neighbor_index)).*dinc_array(neighbor_i
ndex);

            sidelobe_index = find(abs(dheight_array-dheight_ref)<0.5);
            sidelobe_index2 =
unique([sidelobe_index;sidelobe_index+Nlines_buffer]);
            mrm_buffer(~sidelobe_index2) = 0;
            loc_max = imregionalmax(mrm_buffer,8);
            scat_index = find(loc_max);
            sidelobe_index = setdiff(sidelobe_index2,scat_index);

            [sidelobe_az_buffer,sidelobe_r_buffer] = ...
                ind2sub([Nlines_buffer Npixels_buffer],sidelobe_index);
            sidelobe_az = sidelobe_az_buffer+az_min-1;
            sidelobe_r = sidelobe_r_buffer+r_min-1;

          otherwise
            sidelobe_az = zeros(size(ps_az));
            sidelobe_r = zeros(size(ps_r));
        end


        % ------------------------------------------------------------------------
        % Write to file
        % ------------------------------------------------------------------------

        switch orbit
          case 'asc'
            ps_az_av = (Nlines-ps_az)/(r_spacing/az_spacing);
            %ps_az_av = (ps_az-1)/(r_spacing/az_spacing); indicated by Mahmut
            ps_r_av = ps_r-1;
          case 'desc'
            ps_az_av = (ps_az-1)/(r_spacing/az_spacing);
            %ps_az_av = (Nlines-ps_az)/(r_spacing/az_spacing); indicated by
Mahmut
            ps_r_av = Npixels-ps_r;
          otherwise
            error('The orbit (asc/desc) is not specified correctly');
        end

        for k = 1:Nindex
          fseek(ps_plh_inc_fid,(index1(k)-1)*4*8,-1);
          fwrite(ps_plh_inc_fid,[phi_lam_h(k,:) inc_angle_deg(k)]','double');
        end


        switch output_format
          case 1 %wgs84, radar original, radar gis coordinates
            fprintf(ps_ascii_fid,ps_ascii_format1,...
                    [phi_lam_h(:,1) ...
                     phi_lam_h(:,2) ...
                     phi_lam_h(:,3) ...
                     ps_az ...
```

```
                ps_r ...
                ps_az_av ...
                ps_r_av ...
                ps_param ...
                ps_ens_coh ...
                ps_ens_coh_local ...
                ps_stc]');
        case 2 %wgs84, radar original coordinates
          fprintf(ps_ascii_fid,ps_ascii_format1,...
                [phi_lam_h(:,1) ...
                 phi_lam_h(:,2) ...
                 phi_lam_h(:,3) ...
                 ps_az ...
                 ps_r ...
                 ps_param ...
                 ps_ens_coh ...
                 ps_ens_coh_local ...
                 ps_stc]');
        case 3 %wgs84, radar gis coordinates
          fprintf(ps_ascii_fid,ps_ascii_format1,...
                [phi_lam_h(:,1) ...
                 phi_lam_h(:,2) ...
                 phi_lam_h(:,3) ...
                 ps_az_av ...
                 ps_r_av ...
                 ps_param ...
                 ps_ens_coh ...
                 ps_ens_coh_local ...
                 ps_stc]');
        case 4 % radar gis coordinates
          fprintf(ps_ascii_fid,ps_ascii_format1,...
                [ps_az_av ...
                 ps_r_av ...
                 ps_param ...
                 ps_ens_coh ...
                 ps_ens_coh_local ...
                 ps_stc]');
        case 5 % terrafirma S5
               % ps_code = [repmat('PS',Nps_buffer,1) ps_order
repmat('_',Nps_buffer,1) num2str((Nps_count+1:Nps_count+Nps_buffer)','%g')];
          ps_code = (Nps_count+1:Nps_count+Nps_buffer)';
          ps_r_single = (ps_r-1)/2;
          ps_az_single = (ps_az-1)/2;
          ps_std = sqrt(ps_covar(:,25))*1000;
          ps_ts = [ps_defo_tot(:,1:master_index)*1000 zeros(Nps_buffer,1) ...
                   ps_defo_tot(:,master_index+1:Nifgs)*1000];

          fprintf(ps_ascii_fid,ps_ascii_format1,...
                [ps_code,...
                 phi_lam_h(:,2),...
                 phi_lam_h(:,1),...
                 ps_r_single,...
                 ps_az_single,...
                 ps_param(:,1),...
                 ps_param(:,4)*1000,...
                 ps_ens_coh,...
                 ps_std,...
                 ps_ts]');
        case 6 %order, wgs84, radar original, radar gis coordinates
          fprintf(ps_ascii_fid,ps_ascii_format1,...
                [ps_order ...
                 phi_lam_h(:,1) ...
                 phi_lam_h(:,2) ...
                 phi_lam_h(:,3) ...
```

```
                    ps_az ...
                    ps_r ...
                    ps_az_av ...
                    ps_r_av ...
                    ps_param ...
                    ps_ens_coh ...
                    ps_ens_coh_local ...
                    ps_stc]');
        end

        fprintf(ps_defo_fid,ps_defo_format,(1000*ps_defo_tot)');
        fprintf(ps_atmo_fid,ps_atmo_format,(1000*ps_atmo/m2ph)');

        fwrite(ps_sidelobe_fid,[sidelobe_az sidelobe_r]','single');


        fwrite(ps_nodens_fid,ps_data','double');
        Nps_nodens(z,w) = Nps_nodens(z,w)+length(ps_index);
      end

    end %end for g = 1:Ngrid_az
  end %end for w = 1:Nresults
  fclose('all');
end %end for z = 1:Npsc_selections


Nps_atmo_nodens = Nps_nodens(:,1);
switch defo_model_flag
  case 'yes'
    Nps_defo_nodens = Nps_nodens(:,2);
  case 'no'
    Nps_defo_nodens = [];
end
```